- "What are some potential pitfalls or drawbacks of event delegation?"

3.4 Memory management and code architecture

For JavaScript developers aiming to build performant, scalable, and maintainable applications, mastering memory management and modular architecture is essential. While JavaScript's garbage collector automates much of the work behind the scenes, understanding how memory is allocated, retained, and released helps avoid pitfalls like memory leaks and excessive consumption—issues that can silently degrade performance in complex or long-running apps.

On the architectural side, organizing code into reusable, well-structured modules promotes clarity, reduces global scope pollution, and makes large codebases easier to maintain and scale. Modern standards like ES Modules (`import`/`export`) empower developers to encapsulate functionality and enforce clean separation of concerns.

By combining effective memory strategies with architectural patterns, developers can deliver applications that are not only fast and lightweight but also resilient and future-proof.

Table 3.4 is Jane's review guide for memory management and modular architecture. Use it as a checklist to prepare for both conceptual and practical challenges in technical interviews.

Table 3.4 Key features of memory management and code architecture

Topic	Fundamentals	Applied skills	Advanced
Memory & code organization	Describe garbage collection, identify memory leaks, and use modules effectively.	Prevent memory leaks, modularize codebases, and work with ES Modules.	Profile and optimize memory usage, implement code splitting, and enforce scalable architectures.
Data	Use `Map`, `Set`,	Apply	Optimize